

# Project Report for Immersive Technologies Course

Academic Year: 2025-2026

## Virtual Office

*A Multi-User Immersive Collaborative Workspace in VR*

**Riccardo Zannoni**

[riccardo.zannoni@studenti.unitn.it](mailto:riccardo.zannoni@studenti.unitn.it)

256121

**Nicola Cappellaro**

[nicola.cappellaro@studenti.unitn.it](mailto:nicola.cappellaro@studenti.unitn.it)

256119

### Abstract

Remote work and distributed team collaboration have become increasingly common. Existing tools such as Zoom, Google Meet, and Microsoft Teams have **limitations when multiple participants need to share and compare visual content simultaneously**: traditional screen sharing is sequential, allowing only one presenter at a time and creating friction during collaborative tasks.

This project presents **Virtual Office**, a Social Virtual Reality application developed in Unity 6 using the Ubiq networking framework. The system recreates an office environment where up to four remote users can collaborate in real time. Each participant is assigned a personal workstation equipped with a virtual whiteboard, a drawable 2D canvas, and virtual monitors displaying the whiteboards of all other users through Render Textures. **This enables continuous awareness of collaborators' work without requiring screen-sharing handoffs.**

The project is designed for **Meta Quest 3** headsets and controllers. Key technical features include a multi-chunk snapshot synchronization protocol for late-joining users, a peer-property-based workstation assignment mechanism that mitigates race conditions through a timed synchronization window, and a visibility management system that dynamically activates only occupied workstations.



**Testing** with up to four simultaneous users, complemented by a structured evaluation via **SUS** and **SUS Presence Scale**, validated the system's functionality and demonstrated an engaging collaborative experience. Compared to commercial platforms such as Meta Horizon Workrooms, Virtual Office is intentionally **simpler and fully open-source**, making it a valuable educational prototype and a foundation for future research on immersive collaborative workspaces.

## Introduction

The shift towards remote and hybrid work, accelerated by the COVID-19 pandemic, has made distributed team collaboration a daily reality for a large share of the working population. While video conferencing tools such as Google Meet, Zoom, and Microsoft Teams have become standard infrastructure for remote communication, they remain limited in several important respects, particularly when collaboration involves visual content that all participants need to see simultaneously.

The core problem this project addresses is the **sequential nature of screen sharing**: in a group call involving three or more participants, it is only possible for one person to present their screen at a time. In practice, this means that when a team is working on tasks where multiple participants need to share visual information, the session devolves into a series of presenter handoffs. This interaction pattern introduces friction in the collaborative process, interrupts the natural flow of conversation, and weakens the perception of working together in a shared environment.



**Virtual Office** proposes an immersive alternative: a shared virtual office in which up to four remote users can be present simultaneously, each at their own workstation. Every workstation has a drawable whiteboard and a set of virtual monitors that continuously display the whiteboards of the other participants, so that everyone's work is always visible to everyone else, with no handoffs required. The virtual pen and color picker allow basic annotations and diagrams to be created in real time. The environment is designed for the Meta Quest 3 headset, providing a first-person VR experience that aims to reinforce the feeling of sharing a physical space with remote colleagues.

This work builds on theoretical perspectives covered in the Course, particularly the concepts of presence and co-presence. According to [1] Slater and Wilbur (1997), presence comprises **Place Illusion (PI)**, the perceptual sensation of "being there", and **Plausibility Illusion (Psi)**, the credibility of virtual events. The project enhances PI through a persistent, shared spatial environment and bolsters Psi through real-time whiteboard updates that make peer activities perceptually believable. Concurrently, this setup achieves [2] Dourish, P. (2001) concept of **interface transparency**, which dictates that technology becomes more natural when it is less

noticeable. By removing the explicit presenter handoffs and UI friction inherent in traditional video conferencing, the platform allows collaboration to occur directly within the environment, resulting in a seamless, unobtrusive user experience.

Two hypotheses guide this work: first, that a system like Virtual Office can achieve acceptable usability despite the complexity of a multi-user networked VR environment; second, that an immersive VR-based collaboration tool of this kind can increase the sense of presence and immersion compared to traditional video-conferencing methods such as Zoom or Google Meet.

The project deliberately introduces a **simplification**: rather than streaming the real desktop of each participant, users draw on a virtual whiteboard using a pen. This choice makes the implementation tractable within the scope of the course while still demonstrating the core concepts of immersive collaboration, presence and social co-presence. Replacing the whiteboard with a real-time desktop stream is identified as the primary direction for future work.

## 2. Related Work

The problem of remote collaboration has attracted significant research attention, especially following the widespread adoption of hybrid work. Below we review the most relevant systems and research threads.

**Meta Horizon Workrooms** ([3] Meta, 2021) is the most prominent commercial example of a VR-based collaborative office. It allows users to join a shared virtual room, interact through avatars, draw on shared whiteboards, and, critically, bring their physical monitor into VR via a remote desktop stream. Virtual Office shares the high-level goal of Horizon Workrooms but is architecturally simpler, fully open-source, and built on a research-oriented framework rather than a proprietary platform.

**VR whiteboard research:** ([4] Wang et al. 2022) proposed a VR whiteboard system for remote collaboration using natural handwriting and conducted controlled experiments comparing it to traditional and electronic whiteboards. They found that VR whiteboard collaboration efficiency is significantly higher than mouse-based electronic whiteboards and comparable to physical whiteboards. Virtual Office embeds the whiteboard within a broader office simulation and adds the dimension of simultaneous multi-screen visibility.

**ENGAGE XR** and similar platforms ([5] ENGAGE XR, 2025) offer full-featured virtual workspaces with desktop sharing, spatial audio, and virtual whiteboards. These systems corroborate the general finding that VR-based collaboration reduces the sense of distance and meeting fatigue compared to flat video calls. Virtual Office contributes a lightweight, research-transparent implementation of the core multi-screen visibility concept using freely available tools.

## 3. Design

Virtual Office is designed around a central assumption: that the most critical limitation of current video-conferencing tools for small group collaboration is the **inability to see all participants' screens simultaneously**. Below we describe the key design assumptions, architecture, and tools.

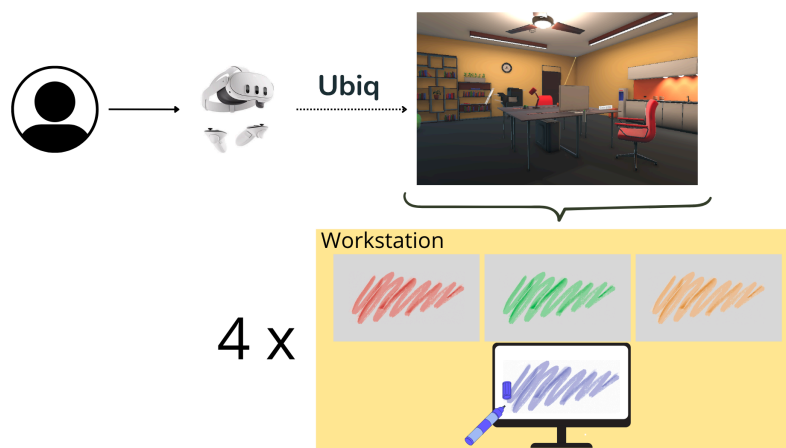
### 3.1 Key Assumptions and Design Rationale

- The target user group is small distributed teams (**2-4 people**) engaged in tasks requiring shared visual content.
- Users have access to a **Meta Quest 3** headset and controllers. The system also runs in flat-screen mode (keyboard + mouse) for development and testing, but the intended experience is immersive VR.
- **Simplifying** the content-sharing medium to a virtual whiteboard (rather than a real desktop stream) is acceptable for a first prototype. The whiteboard captures the core concept of always-on shared visibility with a manageable implementation complexity.
- Users share a single Ubiq room served by a local hosted **server**.

### 3.2 Architecture and Component Communication

The office, in addition to aesthetic decorations, is centered around a central table designed to accommodate four **Workstations** prefabs (Workstation1 through Workstation4), each containing:

- A **Whiteboard**: a flat 2D plane (Texture2D, 1920x1080 RGBA32) rendered on the surface of a monitor mesh. Each Whiteboard has a corresponding RenderTexture used as its mirror.
- Three **Virtual\_Whiteboard\_n** objects (where n indexes the other workstations): these are planes whose material samples the RenderTexture of the Whiteboard belonging to workstation n. This means that Virtual\_Whiteboard\_2 in Workstation1 always shows what is on Whiteboard2, updated in real time with no network overhead beyond what is already used to sync the Whiteboard2 texture.
- A **WhiteboardMarker** (pen prefab): a hierarchy of an empty parent, a Grip child (cylinder), and a Nib child (smaller cylinder). The pen carries an XRGrabInteractable component and can be picked up by the user.



Communication between components follows two paths:

- **Local (in-scene)**: WhiteboardMarker uses a Raycast from its tip. On a hit with a Collider tagged "Whiteboard", it calls whiteboard.SendDraw() to both paint the local texture and send the stroke over the network.
- **Networked (via Ubiq)**: Whiteboard and WhiteboardMarker each have a NetworkContext registered with NetworkScene.

The WorkstationManager listens to RoomClient events (OnJoinedRoom, OnPeerAdded, OnPeerRemoved, OnRoomUpdated) to determine **how many users are connected**. It uses Ubiq peer properties (a key-value store attached to each peer's network identity) to assign each peer a slot number (1-4) and **activates only the workstations corresponding to connected users**. Virtual whiteboards are displayed only on a user's own workstation, showing the boards of the other active slots.

Regarding **voice communication**, Ubiq includes a built-in voice chat module that provides spatial audio between peers out of the box. Virtual Office uses this module without any modifications, so users can speak to each other naturally within the virtual office as soon as they join the room. **No additional configuration** or custom code is required on our side.

### 3.3 Tools

Beyond Unity 6.0 ([6] Unity Technologies. 2024) and Ubiq 1.0.0-pre16 ([7] Friston et al. 2021), the project uses the following tools and packages:

- **XR Interaction Toolkit 3.0.7** (Unity): provides XRGrabInteractable, XR Origin, and controller input abstractions for Meta Quest. ([8] XR Interaction Toolkit. 2024)
- **Visual Studio Code**: development IDE for C# scripts.
- **Office Pack by Nappin** (Unity Asset Store): furniture and prop assets used to furnish the virtual office. ([9] Nappin. 2025)
- **Skybox Series Free by AVIONX** (Unity Asset Store): skybox visible through the office window. ([10] AVIONX. 2022)

### 3.4 Ubiq Modifications and Extensions

No existing Ubiq components were modified. The project started from the Ubiq Demo (XRI) sample scene and extended it by adding four custom MonoBehaviours **scripts** (Whiteboard, WhiteboardMarker, ColorPickerUI, WorkstationManager) that **use Ubiq's public API exclusively**.

### 3.5 Application Walk-through

This section describes the user experience from first launch to active collaboration.

**Joining the office:** when a user starts the application, they are placed in the room in front of the Ubiq panel. The first user has to create the room through the panel, while the other users have to join the created room using the room code. Once inside the room, only the workstations corresponding to currently connected users are active and visible; the rest of the office furniture is always visible to provide context. A user's workstation consists of a personal whiteboard positioned in front of them and the virtual monitors located above it.



**Drawing:** a virtual pen is located beside the whiteboard. The user grabs it by pressing the grip trigger on their Quest 3 controller (VR mode) or the right mouse button (flat-screen mode). Once held, moving the pen tip towards the whiteboard surface causes the nib to draw on the whiteboard.

**Changing color:** a button beside the workstation allows the users to open a floating color picker panel. The panel exposes three sliders (Hue, Saturation, Value) and an optional hex code input field. A color preview swatch shows the current color in real time. An Eraser button sets the color to white. The UI synchronizes automatically with the color of the pen currently in the user's hand.

**Viewing other participants:** the virtual monitors above each user's whiteboard display the whiteboards of the other connected participants as live Render Textures. They are read-only: users can only draw on their own whiteboard.



**Late Join:** when a new user joins a room where other participants have already drawn on their whiteboards, the new peer automatically sees all drawings that were created before they entered the room.

## 4. Implementation

### 4.1 Development Process

Development began from the Ubiq Demo (XRI) sample scene, which provided a ready-made networked environment with avatars, voice chat, and basic interaction support. We stripped out all outdoor decorations, terrain, and placeholder objects, leaving only the core Ubiq infrastructure. On top of this clean base we built a simple enclosed room using flat-plane walls, and progressively furnished it with the office assets described in Section 3.3: desks, chairs, monitors, and decorative props from the Office Pack ([9] Nappin. 2025). A local Ubiq server connection profile was also configured at this stage to support testing

The first milestone was implementing a functional virtual pen. We created the WhiteboardMarker prefab, which is for the creation of the pens, and attached an XRGrabInteractable component to enable picking it up with the Quest 3 controller or the mouse. In the initial prototype the pen had no surface awareness: strokes were drawn in mid-air at an arbitrary depth. We then extended the implementation by

adding a short Raycast from the nib along its local up axis; when the ray intersects a Collider tagged "Whiteboard", the UV coordinates of the hit point are converted to pixel coordinates and painted onto the whiteboard texture ([Ref: A.1](#)). Network synchronization of the pen's position and rotation was added at the same stage, so remote peers can observe each other's pen movements in real time. ([Ref: A.2](#))

With a working single-user pen, we assembled the full workstation layout: each of the four Workstation prefabs received one drawable Whiteboard (a 1920×1080 Texture2D plane mapped onto the monitor mesh) and three Virtual\_Whiteboard planes whose materials sample the RenderTextures of the other three whiteboards. This gives every user a persistent, always-on view of their collaborators' screens without any additional network traffic beyond what is already used to sync the whiteboard textures. ([see Section 4.2 below](#))

## 4.2 Custom Scripts

Four additional C# scripts were developed to cover the remaining feature requirements:

**WhiteboardMarker.cs:** as mentioned before, this script implements a networked XR marker that allows users to draw on a whiteboard, synchronizing both marker movements and drawing actions across all connected peers. ([Ref: A.1 and A.2](#))

**Whiteboard.cs.** Beyond handling real-time stroke synchronization between peers, this script implements a late-join protocol. When a new user enters a room where others have already drawn on their whiteboards, the script coordinates a snapshot transfer: the existing peer designated as master ([Ref: A.3](#)) encodes its whiteboard texture as a JPEG compressed image, splits it into chunks of 24 KB, and sends them over the network ([Ref: A.4](#)). The joining peer reassembles the chunks and applies the result, so it immediately sees the current state of the whiteboard rather than an empty surface.

**WorkstationManager.cs** keeps the virtual office layout consistent with the number of connected users. It listens to Ubiq room events and sequentially activates workstations 1 through 4 based on the total peer count ([Ref: A.6](#)). At join time, each peer runs a coroutine that waits briefly for existing peers' properties to propagate over the network, then reads which slots are already taken and claims the first free one, writing it to its own peer property ([Ref: A.5](#)). The script also controls which **Virtual\_Whiteboard** panels are displayed: they are shown exclusively on the local player's own workstation, so each user sees only the screens of peers who are currently connected.

**ColorPickerUI.cs.** This script implements an in-scene HSV color picker panel that lets users change the ink color of the pen they are currently holding. The panel exposes sliders for Hue, Saturation, and Value, a real-time color preview swatch, an optional hex code input, and an eraser button that sets the color to white. When a marker is grabbed, it registers itself with the color picker via a lightweight singleton reference, and the UI immediately syncs its sliders to that marker's current color, so the panel always reflects the color of the pen actually being held. ([Ref: A.7](#))



### 4.3 Challenges and Solutions

**Workstation slot race condition.** During early multi-user tests we observed that every peer that joined the room was assigned to workstation 1, regardless of how many users were already connected. The root cause was a race condition: when a new peer joined, it read the slot assignments of the other peers before their Ubiq peer properties had propagated across the network, so all slots appeared free. The fix was to introduce a short coroutine delay before the slot-reading logic executes, giving the network time to deliver the existing peers' properties before the new peer makes its assignment decision.

**Dynamic workstation visibility.** Initially, all workstations were visible regardless of the number of connected users, resulting in unused work areas being displayed in the virtual environment. To improve clarity and reduce visual clutter, the workstation manager was modified to dynamically adjust the number of visible workstations based on the number of users currently present in the room.

**Discontinuous strokes.** In the first networked drawing implementation, remote peers received only the current touch position for each frame, which produced isolated dots rather than connected strokes. The fix was to include the previous touch position (`last_point`) in every network message, enabling the receiving peer to linearly interpolate between the two points and fill in the gap. This produces smooth, continuous strokes on all clients regardless of network tick rate.

**Dynamic pen-color binding.** The initial color picker implementation had no way to determine which pen should receive a color change at runtime, since multiple pens can exist in the scene and any of them could be in the user's hand. We solved this by introducing a marker ownership system: when a pen is grabbed, it registers itself with the `ColorPickerUI` singleton as the active marker; when it is released, it unregisters. All color changes made through the UI are then applied exclusively to the currently registered marker, making the binding fully dynamic.

### 4.4 Environment Finalization



Once all interactive features were working, we finalized the visual appearance of the office. We added a variety of **decorative objects** typical of a real workspace: bookshelves, plants and desk accessories, to increase the sense of **presence** and make the environment feel inhabited. The **lighting setup** combines two approaches: real-time lights, which update dynamically every frame and can respond to changes in the scene, and baked lights, whose illumination is pre-computed and stored in lightmap textures at build time. Finally, a **Skybox** from the Skybox Series Free asset pack was applied to the background visible through the office window, simulating an outdoor sky and reinforcing the sense of being inside a real building.

## 5. Evaluation

Due to the unavailability of Meta Quest 3 headsets during the development period, the formal evaluation was conducted in flat-screen mode (keyboard and mouse input on a PC), which represents a significant limitation that must be kept in mind when interpreting the results. A structured experiment with multiple independent participants using the intended VR hardware would be essential to draw statistically significant conclusions and is identified as a priority for future work.

### 5.1 Research Hypotheses

The evaluation was guided by two research hypotheses:

- **H1 (Usability):** Virtual Office achieves acceptable usability even in flat-screen mode, given its simple interaction model.
- **H2 (Presence):** Sense of presence, measured via the SUS Presence Scale, will be limited in flat-screen mode due to the absence of a head-mounted display and natural hand interaction.

### 5.2 Experimental Design

**Design:** A single-condition, within-subjects descriptive study was conducted. No independent variable was manipulated; instead, the study aimed to characterize baseline usability and presence in the flat-screen deployment mode. This choice reflects the hardware constraints described above.

**Participants:** 4 participants at the same time (all together in the same virtual office) took part in the study. All had prior experience with desktop applications; none had experience with social VR systems.

**Independent, Dependent, and Control Variables:** Given the single-condition design, there was no manipulated independent variable. The dependent variables were: (1) perceived system usability, measured via the System Usability Scale (SUS; Brooke, 1996 [11]), and (2) sense of presence, measured via the SUS Presence Scale (Usoh et al., 2000 [12]). Control variables included the hardware platform (identical PC setup for all participants) and the task set.

**Tasks:** Each participant joined a shared Virtual Office room, tried to draw something on the whiteboard, watch the others' drawing in the virtual whiteboards, and communicate with voice using the microphone. At the end each participant filled the questionnaires individually. Total session duration was approximately 20 minutes.

### 5.3 Functional Testing

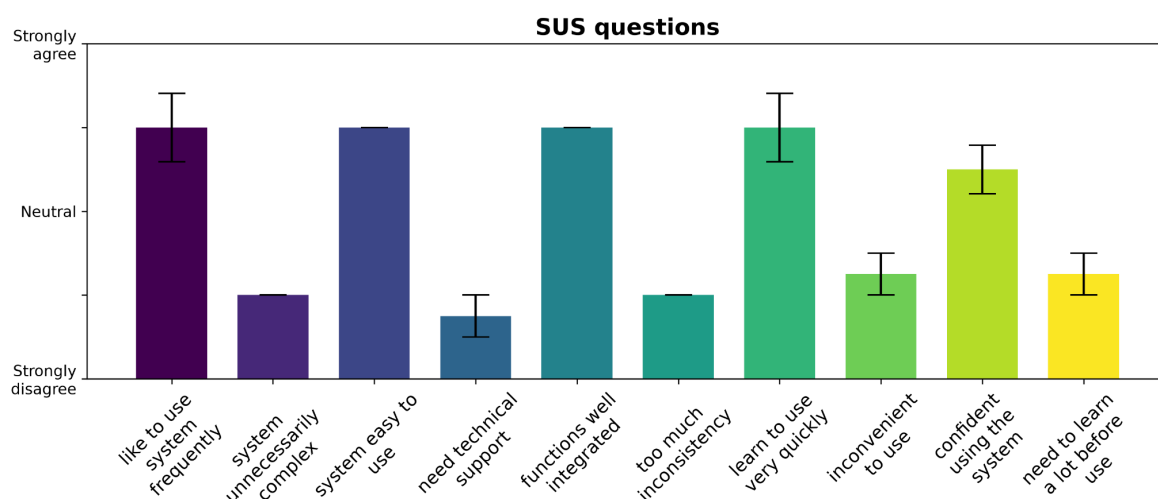
Prior to the usability study, the following functional scenarios were verified:

- **Single user:** correct whiteboard rendering, pen grab/release, drawing, color picker.
- **Two-user session:** real-time stroke synchronization between peers, correct Virtual\_Whiteboard mirroring.
- **Three and four-user sessions:** correct workstation activation/deactivation, correct slot assignment, correct Virtual\_Whiteboard visibility per workstation.
- **Late join:** new peer correctly receives snapshot of existing whiteboard content.

All scenarios passed functional verification. One known issue emerged: when a user disconnects, the workstation that is deactivated is the last one added rather than the one belonging to the disconnecting user. This is a limitation of the current disconnect handling logic in WorkstationManager ([see Section 6](#)).

### 5.4 Usability Results (SUS)

To formally test our **H1 hypothesis** we used SUS questionnaires. The results are shown below:



We obtained a mean score of **73.125**. This score falls above the established average threshold of 68, placing Virtual Office in the "good" usability range: we can say that hypothesis H1 is therefore supported.

The most recurring negative feedback concerned the difficulty of controlling the pen and drawing with a mouse: this limitation is inherent to the flat-screen testing mode and is expected to improve significantly in a VR deployment with Meta Quest 3 controllers, which would likely push the score further upward.

## 5.5 Presence Results (SUS Presence Scale)

To test the **H2 hypothesis** we used the SUS Presence Scale (6 items, with a scale from 1 to 7).

The mean presence score of **3.42/7** is moderate, consistent with Hypothesis H2. The score reflects the expected reduction in presence caused by the flat-screen deployment: without a head-mounted display, spatial immersion is inherently limited and participants remained largely aware of their physical desktop environment.

| Participant | Mean score  |
|-------------|-------------|
| P1          | 3.50        |
| P2          | 3.17        |
| P3          | 4.23        |
| P4          | 3.17        |
| <b>Mean</b> | <b>3.42</b> |

A deployment on Meta Quest 3 headsets is expected to substantially increase presence scores, as the Place Illusion and Plausibility Illusion components identified by Slater and Wilbur [1] would both be significantly enhanced by the first-person VR perspective and natural controller-based interaction.

## 5.6 Informal Usability Observations

In addition to the questionnaires, the following qualitative observations were recorded:

- The overall experience was described as functional and easy to understand, but lacking the immersive quality that would be expected from a VR deployment.
- The always-on visibility of all collaborators' whiteboards was perceived as the most valuable feature
- Pen control via mouse was consistently described as awkward: all participants noted that drawing precision was significantly lower than expected.
- The color picker UI was rated as intuitive and the HSV sliders were easy to use.

A more formal experiment, had hardware been available, would have measured:

1. task completion time and drawing accuracy on a shared annotation task (behavioral), and
2. more meaningful SUS and SUS Presence scores, since the intended immersive VR experience could not be reproduced in flat-screen mode.

## 6. Discussion and Conclusions

### 6.1 Limitations

**Whiteboard as a simplification.** The most significant limitation is the substitution of the real desktop with a drawable whiteboard. The original problem, not being able to see everyone's actual screen at once, is only partially solved: the whiteboard allows users to draw diagrams and annotations, but does not show the actual computer screen of each participant. Replacing the whiteboard with a real-time desktop video stream would fully resolve the initial problem.

**Disconnect handling:** when a user disconnects, the WorkstationManager deactivates the last workstation in terms of slot number rather than the one

belonging to the disconnecting user. This creates a visual inconsistency. A correct implementation would map each peer UUID to its slot at the time of connection and use that mapping on disconnect.

**Mouse precision:** the pen is difficult to control using a mouse, as the flat-screen input mode does not replicate the physical motion of handwriting. This is not a concern for VR use but limits the accessibility of the application for users without headsets.

**Fixed four-user limit:** the system supports exactly four workstations. Extending to more users would require a more flexible workstation layout and a generalized slot-assignment protocol.

**Limited evaluation conditions:** the formal evaluation was conducted in flat-screen mode due to the unavailability of Meta Quest 3 headsets, which prevented participants from experiencing the intended VR deployment. As a result, both usability and presence scores are expected to underestimate the quality of the experience in the target hardware configuration.

## 6.2 Lessons Learned

Developing this multi-user VR application highlighted the importance of handling networking concerns, such as synchronization and late joins, from the beginning of the design process. A clear communication structure between peers helped simplify development and debugging.

Another important lesson was to build the system incrementally. Implementing and testing one feature at a time made it easier to identify and solve problems before adding new functionality.

Finally, testing showed that user immersion depends not only on networking features but also on details such as environment quality, spatial audio, and natural object interactions, all of which contribute significantly to the overall experience.

## 6.3 Future Work

The primary future development is the replacement of the virtual whiteboard with a real-time stream of each participant's physical monitor. This would completely resolve the original problem and is technically feasible using Unity's screen capture or external video streaming libraries. Other improvements include:

- Proper disconnect handling: mapping peer UUIDs to slots and deactivating the correct workstation on disconnect.
- Extending the user limit beyond four, with a dynamic workstation layout.
- Conducting a formal user study on Meta Quest 3 headsets with an adequate number of independent participants, to obtain statistically reliable presence and usability scores under the intended VR conditions.
- Implementing a whiteboard clear button per-workstation, visible in the UI, rather than relying solely on painting over with white.

## 6.4 Conclusions

Virtual Office demonstrates that the core concept of a shared VR office with always-on multi-screen visibility is both technically feasible with freely available tools (Unity 6, Ubiq, XR Interaction Toolkit) and perceptually effective. Testing with up to four simultaneous users, including a structured evaluation via SUS and SUS Presence Scale, confirmed the system's functionality and produced an experience described as collaborative and engaging. Furthermore, the design goal of eliminating the presenter handoff was successfully achieved. The whiteboard simplification is the main gap between the prototype and a fully practical tool; bridging it through real-time desktop streaming is a clear and well-motivated direction for future work.

## Group Contribution Statement

The project was developed by two people working in parallel on a single shared Unity scene. Both team members contributed to all aspects of the project throughout its development. Specific contributions:

**Riccardo Zannoni:** office design and workstation positioning

**Nicola Cappellaro:** lighting setup and ColorPicker UI implementation

## Code Appendix

Selected key code snippets are presented below. The complete source code is submitted as a ZIP archive.

### A.1 WhiteboardMarker.cs: Drawing with Interpolation

It shows exactly how the ray's point of contact is mapped from UV coordinates to texture pixels, with clamping to prevent out-of-bounds errors.

```
if (Physics.Raycast(_tip.position, _tip.up, out _touch, _drawDistance)){
    if (_touch.transform.CompareTag("Whiteboard")){
        _touchPos = new Vector2(_touch.textureCoord.x, _touch.textureCoord.y);
        int x = (int)(_touchPos.x * _whiteboard.textureSize.x - (_penSize / 2));
        int y = (int)(_touchPos.y * _whiteboard.textureSize.y - (_penSize / 2));
        x = Mathf.Clamp(x, 0, (int)_whiteboard.textureSize.x - _penSize);
        y = Mathf.Clamp(y, 0, (int)_whiteboard.textureSize.y - _penSize);

        if (_touchedLastFrame){
            for (float f = 0f; f <= 1.00f; f += 0.01f){
                int lerpX = Mathf.Clamp((int)Mathf.Lerp(_lastTexX, x, f), 0, (int)_whiteboard.textureSize.x - _penSize);
                int lerpY = Mathf.Clamp((int)Mathf.Lerp(_lastTexY, y, f), 0, (int)_whiteboard.textureSize.y - _penSize);
                _whiteboard.texture.SetPixels(lerpX, lerpY, _penSize, _penSize, _colors);
            }
            _whiteboard.texture.Apply();
            _whiteboard.UpdateRenderTexture();
            _whiteboard.SendDraw(x, y, _lastTexX, _lastTexY, true, _penSize, _currentColor);
        }
    }
}
```

### A.2 WhiteboardMarker.cs: Update and Networking

Show the owner/non-owner pattern: only the peer that owns the marker sends position and rotation data, while the other peers receive this data and apply the mechanism locally that allows them to observe the movement of other users' pens in real time.

```
void Update(){
    if (!_isOwner) return;

    Draw();

    context.SendJson(new MarkerMessage{
        position = transform.position,
        rotation = transform.rotation
    });
}

public void ProcessMessage(ReferenceCountedSceneGraphMessage message){
    if (!_isOwner) return;

    var data = message.FromJson<MarkerMessage>();
    transform.position = data.position;
    transform.rotation = data.rotation;
}
```

### A.3 Whiteboard.cs: Master election

It demonstrates how the peer responsible for sending the snapshot is determined deterministically (the peer with the “smallest” UUID among the existing ones),

thereby preventing redundant transmissions by all peers.

```
private void OnPeerAdded(IPeer newPeer){
    if (_roomClient == null) return;
    if (!gameObject.activeInHierarchy || !enabled) return;

    string myUuid = _roomClient.Me.uuid;
    bool iAmMaster = true;

    foreach (var peer in _roomClient.Peers){
        if (peer.uuid == newPeer.uuid) continue;
        if (string.Compare(peer.uuid, myUuid, StringComparison.Ordinal) < 0){
            iAmMaster = false;
            break;
        }
    }

    if (!iAmMaster) return;
    if (IsTextureBlank()){
        Debug.Log("[Whiteboard] I'm master but the whiteboard is blank: snapshot not sent.");
        return;
    }

    StartCoroutine(SendSnapshotAfterDelay(0.5f));
}
```

#### A.4 Whiteboard, Snapshot Send with Chunking

Shows the encoding process: JPEG → base64 → fragmentation into numbered chunks ready for transmission.

```
private void SendSnapshot(){
    byte[] jpgBytes = texture.EncodeToJPG(85);
    string base64 = Convert.ToBase64String(jpgBytes);

    int totalChunks = Mathf.CeilToInt((float)base64.Length / CHUNK_BYTES);
    Debug.Log($"[Whiteboard] Send snapshot: {jpgBytes.Length / 1024} KB → {totalChunks} chunk/s");

    for (int i = 0; i < totalChunks; i++){
        int start = i * CHUNK_BYTES;
        int length = Mathf.Min(CHUNK_BYTES, base64.Length - start);

        context.SendJson(new SnapshotChunkMessage{
            chunkIndex = i,
            totalChunks = totalChunks,
            base64Data = base64.Substring(start, length)
        });
    }
}
```

#### A.5 WorkstationManager.cs: Slot Assignment Coroutine

This demonstrates the complete slot-claiming mechanism: waiting for peer properties to propagate, reading occupied slots, selecting the first available slot, and writing persistently to one's own peer property, with a retry if no slots are available.

```

private IEnumerator ClaimSlotCoroutine(){
    yield return new WaitForSeconds(0.8f);
    bool[] occupied = new bool[5]; // Index 1-4, 0 ignored

    foreach (var peer in roomClient.Peers){
        string val = peer[SLOT_KEY];
        if (!string.IsNullOrEmpty(val) && int.TryParse(val, out int taken) && taken >= 1 && taken <= 4){
            occupied[taken] = true;
            Debug.Log($"[WorkstationManager] Peer {peer.uuid} takes slot {taken}");
        }
    }

    for (int slot = 1; slot <= 4; slot++){
        if (!occupied[slot]){
            localPlayerNumber = slot;
            roomClient.Me[SLOT_KEY] = slot.ToString();
            Debug.Log($"[WorkstationManager] Slot assigned: Player {localPlayerNumber}");
            RefreshVisibility();
            yield break;
        }
    }

    Debug.LogWarning("[WorkstationManager] No available slots, retrying in 2s...");
    yield return new WaitForSeconds(2f);
    StartCoroutine(ClaimSlotCoroutine());
}

```

## A.6 WorkstationManager.cs: Adaptive Workstation and Whiteboard Management

It demonstrates how the number of connected peers dynamically determines how many workstations are active, and how the Virtual\_Whiteboard panels for other users are only visible on the local player's workstation and only for peers who are

```

private void RefreshVisibility(){
    if (roomClient == null) return;
    int activeCount = Mathf.Clamp(roomClient.Peers.Count() + 1, 1, 4);

    if (localPlayerNumber != 0 && activeCount == lastActiveCount && localPlayerNumber == lastLocalPlayerNumber) return;

    lastActiveCount = activeCount;
    lastLocalPlayerNumber = localPlayerNumber;
    Debug.Log($"[WorkstationManager] Refresh → Users: {activeCount} | LocalPlayer: {localPlayerNumber}");

    for (int i = 0; i < workstations.Length; i++){
        if (workstations[i] == null){
            Debug.LogWarning($"[WorkstationManager] workstations[{i}] is null!");
            continue;
        }

        int workstationNumber = i + 1;
        bool shouldBeActive = workstationNumber <= activeCount;
        workstations[i].SetActive(shouldBeActive);

        if (shouldBeActive)
            UpdateVirtualWhiteboards(workstations[i], workstationNumber, activeCount);
    }
}

private void UpdateVirtualWhiteboards(GameObject workstation, int ownWorkstationNumber, int activeCount){
    bool isMyWorkstation = (localPlayerNumber != 0 && ownWorkstationNumber == localPlayerNumber);

    for (int j = 1; j <= 4; j++){
        if (j == ownWorkstationNumber) continue;

        string vwName = $"Virtual_Whiteboard_{j}";
        Transform vwTransform = workstation.transform.Find(vwName) ?? FindDeepChild(workstation.transform, vwName);

        if (vwTransform == null){
            if (j <= activeCount)
                Debug.LogWarning($"[WorkstationManager] '{vwName}' not found " + $"in Workstation{ownWorkstationNumber}.");
            continue;
        }

        bool show = isMyWorkstation && j <= activeCount;
        vwTransform.gameObject.SetActive(show);
    }
}

```

## A.7 Colorpickerui.cs: Marker owner

This demonstrates the singleton pattern through which the WhiteboardMarker “links” to the color picker when it is grabbed: the UI reads the current HSV value of the grabbed marker and updates the sliders accordingly, so that the panel always reflects the color of the pen currently being held.



```
public void RegisterMarker(WhiteboardMarker marker){
    _marker = marker;

    if (_marker != null){
        Color.RGBToHSV(_marker.GetColor(), out _h, out _s, out _v);
        _suppressCallbacks = true;
        ApplyHSVToSliders();
        _suppressCallbacks = false;
        RefreshUI();
    }

    Debug.Log($"[ColorPickerUI] Marker registered: {marker?.name ?? "null"}");
}
```

## References

- [1] Slater, M., & Wilbur, S. (1997). A framework for immersive virtual environments (FIVE): Speculations on the role of presence in virtual environments. *Presence: Teleoperators and Virtual Environments*, 6(6), 603-616.
- [2] Dourish, P. (2001). *Where the action is: The foundations of embodied interaction*. MIT Press.
- [3] Meta. (2021). Introducing Horizon Workrooms: Remote Collaboration Reimagined.  
<https://about.fb.com/news/2021/08/introducing-horizon-workrooms-remote-collaboration-reimagined/>
- [4] Wang, Y., et al. (2022). A Virtual Reality Whiteboard System for Remote Collaboration Using Natural Handwriting. *Electronics*, 11(24), 4152.  
<https://doi.org/10.3390/electronics11244152>
- [5] ENGAGE XR. (2025). Remote VR/AR Collaboration.  
<https://engagevr.io/remote-vr-ar-collaboration/>
- [6] Unity Technologies. (2024). Unity 6 User Manual.  
<https://docs.unity3d.com/6000.0/Documentation/Manual/UnityManual.html>
- [7] Friston, S., Congdon, B. J., Swapp, D., Gorringer, C., Dungworth, N., Steer, C., & Steed, A. (2021). Ubiq: A System to Build Flexible Social Virtual Reality Experiences. In *Proceedings of the 27th ACM Symposium on Virtual Reality Software and Technology (VRST '21)*. <https://doi.org/10.1145/3489849.3489871>
- [8] Unity Technologies. (2024). XR Interaction Toolkit (version 3.0).  
<https://docs.unity3d.com/Packages/com.unity.xr.interaction.toolkit@3.0/manual/index.html>
- [9] Nappin. (2025). *Office Pack - Free* [Unity Asset Store].  
<https://assetstore.unity.com/packages/3d/props/interior/office-pack-free-258600>
- [10] AVIONX. (2022). *Skybox Series Free* [Unity Asset Store].  
<https://assetstore.unity.com/packages/2d/textures-materials/sky/skybox-series-free-103633>
- [11] Brooke, J. (1996). SUS: A "quick and dirty" usability scale. In P. W. Jordan et al. (Eds.), *Usability Evaluation in Industry* (pp. 189-194). Taylor & Francis.
- [12] Usoh, M., Catena, E., Arman, S., and Slater, M. (2000). Using presence questionnaires in reality. *Presence: Teleoperators and Virtual Environments*, 9(5), 497–503.